

Computational Logic

Satisfiability

Sandra Alves - DCC/FCUP

2026/27

We have define:

- Normal forms: negative, disjunctive and conjunctive
- Disjunctive normal forms and satisfiability
- Conjunctive normal forms and tautologies

Horn's formulas

Horn's formulas

A *Horn's formula* is a formula in conjunctive normal form in which in each disjunction (clause) there is at most one positive literal.

$$p \wedge \neg q \wedge (q \vee \neg p)$$

$$(\neg p \vee \neg q \vee \neg s \vee p) \wedge (\neg q \vee \neg r \vee p) \wedge (\neg p \vee \neg s \vee s)$$

$$(\neg p \vee \neg q \vee \neg s) \wedge (\neg q \vee \neg r \vee p) \wedge s$$

In a Horn formula, the disjunctions $\neg p_1 \vee \dots \vee \neg p_n \vee p$ can also be written as:

$$(p_1 \wedge \dots \wedge p_n) \rightarrow p$$

or if p does not exist (or is **F**):

$$(p_1 \wedge \dots \wedge p_n) \rightarrow \mathbf{F}$$

or if the p_i do not exist:

$$\mathbf{V} \rightarrow p$$

Not all formulas have an equivalent Horn formula!

But...

To determine whether a Horn formula of propositional logic is **satisfiable** we can use an efficient algorithm!

Satisfiability of a Horn formula

Consider the formula $p \wedge \neg q \wedge (q \vee \neg p)$:

- start by placing in a line the propositional variables that occur in the formula and the formula. Ex:

$$\frac{p \mid q \parallel p \wedge \neg q \wedge (q \vee \neg p)}{\quad}$$

- if any of the propositional variables is one of the elements of the conjunction assign the value **V** to that variable (why?).

Ex:

$$\frac{p \mid q \parallel p \wedge \neg q \wedge (q \vee \neg p)}{\mathbf{V} \mid \quad \parallel}$$

Satisfiability of a Horn formula

- With this information, fill in the table as if you were constructing the truth table (for this line), analyzing each disjunction to determine if, for it to be true, more values can be determined for the propositional variables:

p	q	$p \wedge \neg q \wedge (q \vee \neg p)$					
V		F					

- In this case, q must be **V** and so this can be added:

p	q	$p \wedge \neg q \wedge (q \vee \neg p)$					
V	V	F					

Satisfiability of a Horn formula

- And repeating this step again, we get:

p	q	$p \wedge \neg q \wedge (q \vee \neg p)$					
V	V	F					

Continue until nothing more can be added.

- if in the previous step **F** is assigned to one of the elements of the conjunction, the formula also has the value **F** and is not satisfiable. Otherwise we can assign the value **V** to the formula if we assign **F** to the remaining propositional variables.

In the example we are considering, the formula has the value **F** and therefore is not satisfiable.

Satisfiability of a Horn formula

Apply the previous algorithm to

$$p \wedge (\neg p \vee q) \wedge \neg r$$

Satisfiability

Knowing whether a formula of propositional logic is **satisfiable** is a central problem in Computer Science since many problems in other areas can be expressed in terms of it:

- optimization: planning, scheduling, etc.
- combinatorics: graph coloring, etc.
- first-order logic theories: linear programming, real-number arithmetic, bit strings, pointers, etc. (SMT solvers).

But if a formula is not in disjunctive normal form or is not in a class for which satisfiability is easy to determine, is constructing a truth table the only way?

In the worst case, **YES**... that is, no algorithm for satisfiability is known that is not exponential in the number of variables and connectives in the formula! (SAT is NP-complete).

In the general case, we can find algorithms that determine the satisfiability of a formula more efficiently than the **brute force** construction of the truth table. The general idea is to partially construct an assignment that satisfies the formula ... instead of generating a complete assignment (truth table row) and checking whether the formula satisfies it!.

This process is especially simplified if the formulas are in conjunctive normal form (CNF). This can be represented compactly using the notion of a **clause**.

Clauses

A **clause** is a disjunction of literals $l_1 \vee l_2 \vee \dots \vee l_n$, $n \geq 0$. A clause can be represented by a set of literals. If $n = 0$ we say that the clause is **empty** and corresponds to **F**. If $n = 1$ we say that the clause is **unitary**.

Example

$p \vee \neg q \vee \neg p \vee s$ can be represented by $\{p, \neg q, \neg p, s\}$.

Any formula of propositional logic in FNC can be represented by a set of clauses.

$$\neg p \wedge (q \vee r \vee q) \wedge (\neg r \vee \neg s) \wedge (p \vee s) \wedge (\neg q \vee \neg s)$$

corresponds to the set of clauses:

$$\{\{\neg p\}, \{q, r\}, \{\neg r, \neg s\}, \{p, s\}, \{\neg q, \neg s\}\}$$

Complementary literals: given a literal l , the *complementary literal* $\tilde{l}$ is defined by:

$$\tilde{l} = \begin{cases} \neg l, & \text{if } l \text{ is a variable (positive)} \\ p, & \text{if } l \text{ is of the form } \neg p \text{ (negative)} \end{cases}$$

For example, $\neg \tilde{p} = p$ and $\tilde{\tilde{p}} = \neg p$.

As we have seen, a clause is a **tautology** if it contains a pair of complementary literals p and $\neg p$.

These clauses can be **removed** from the set without altering satisfiability.

Davis-Putnam Algorithm

Initial version from 1960 that is still the basis of many of the most efficient algorithms today:

<http://www.satlive.org/index.jsp> or Competition

<http://www.satcompetition.org/>.

The basic idea of the algorithm is to consider the possible truth values for each variable and simplify the formula according to these assignments until it is possible to conclude whether it is satisfiable or not.

Unit Propagation

Let S be a set of clauses. A set S' is obtained from S by **unit propagation** if S' is obtained from S by repeating the following transformation: if S contains a unit clause l then:

1. remove from S all clauses of the form $l \vee C'$
2. replace in S each clause of the form $\tilde{l} \vee C'$ with the clause C' .

Davis-Putnam algorithm

Considers the following set of clauses:

$$\{p_1, \neg p_1 \vee \neg p_2, p_3 \vee p_2, \neg p_7 \vee p_2, \neg p_3 \vee p_4, \neg p_3 \vee p_5, \\ \neg p_4 \vee \neg p \vee q, \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p \vee p_7, \neg r \vee p_7\}$$

(1)

Applying to p_1

$$\{\neg p_2, p_3 \vee p_2, \neg p_7 \vee p_2, \neg p_3 \vee p_4, \neg p_3 \vee p_5, \neg p_4 \vee \neg p \vee q, \\ \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p \vee p_7, \neg r \vee p_7\}$$

(2)

Davis-Putnam algorithm

Applying propagation to $\neg p_2$

$$\{p_3, \neg p_7, \neg p_3 \vee p_4, \neg p_3 \vee p_5, \neg p_4 \vee \neg p \vee q, \\ \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p \vee p_7, \neg r \vee p_7\}$$

(3)

After applying the propagation at p_3 and $\neg p_7$, we have:

$$\{p_4, p_5, \neg p_4 \vee \neg p \vee q, \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p, \neg r\}$$

(4)

Davis-Putnam Algorithm

Propagating to $p_4, p_5, \neg r$ and p we get:

$$\{q, \neg p_6, \neg q \vee p_6\} \quad (5)$$

And finally propagating to q and $\neg p_6$:

$$\{\mathbf{F}\} \quad (6)$$

the initial set of clauses is unsatisfiable (and in this case using only unitary propagation).

Davis-Putnam Algorithm

DLL(S)

input: set of clauses S

output: satisfiable or unsatisfiable

S := propagate(S)

if S empty then return satisfiable

if S contains F then return unsatisfiable

/ := select_literal(S)

if DLL(S $\cup$ {/}) = satisfiable

then return satisfiable

else return DLL(S $\cup$ { $\tilde{/}$ })

Davis-Putnam Algorithm

The function `select_literal` returns a literal of the clause. This function can be considered a parameter of the algorithm, since an appropriate choice of literal can make the algorithm more efficient. Possible criteria are, choose a variable that:

- occurs more often;
- the product of the occurrences of l and $\tilde{l}$ is maximum;
- occurs most often in clauses of minimum size;
- etc...

Application of the Algorithm

$$S = \{\neg p \vee \neg q, \neg p \vee q, p \vee \neg q, p \vee q\} \quad (7)$$

Since there are no unit clauses, we have to select a literal, for example, $\neg p$.

By propagation:

$$\begin{aligned} \{\neg p \vee \neg q, \neg p \vee q, p \vee \neg q, p \vee q, \neg p\} &\Rightarrow \\ &\{\neg q, q\} \Rightarrow \{\mathbf{F}\} \end{aligned} \quad (8)$$

Application of the Algorithm

We still have to consider $S \cup \{p\}$. In this case the propagation is:

$$\begin{aligned} \{\neg p \vee \neg q, \neg p \vee q, p \vee \neg q, p \vee q, p\} &\Rightarrow \\ \{\neg q, q\} &\Rightarrow \{\mathbf{F}\} \end{aligned} \tag{9}$$

And the algorithm returns `unsatisfiable`.